

## Spring Boot CRUD 완성 프로젝트

Java + Kotlin · JPA · DTO · Entity · Repository · Service · REST Controller · MariaDB

### 이 문서에서 고친 곳

- Java 예제가 Lombok(@Getter·@Setter 등)을 쓰는데 의존성 안내가 없어, 그대로 따라 하면 컴파일 오류가 납니다. 아래 의존성을 먼저 넣습니다.

```
<!-- pom.xml 의 <dependencies> 안에 -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

xml

```
// build.gradle.kts 를 쓰는 Java 프로젝트라면
compileOnly("org.projectlombok:lombok")
annotationProcessor("org.projectlombok:lombok")
```

kotlin

### 1. CRUD와 REST API

| CRUD   | SQL    | HTTP   | 예제                           | 기능       |
|--------|--------|--------|------------------------------|----------|
| Create | INSERT | POST   | /api/members                 | 회원 등록    |
| Read   | SELECT | GET    | /api/members, /api/members/1 | 전체/단건 조회 |
| Update | UPDATE | PUT    | /api/members/1               | 회원 수정    |
| Delete | DELETE | DELETE | /api/members/1               | 회원 삭제    |

전체 흐름: Client → Controller → Service → DTO ↔ Entity → Repository → Spring Data JPA/Hibernate → MariaDB

### 2. 프로젝트 폴더 구조

#### Java

```
member-java/
├─ pom.xml
├─ mvnw / mvnw.cmd
├─ src/main/
│   └─ java/com/example/member/
│       ├── MemberApplication.java
│       ├── controller/MemberController.java
│       ├── service/MemberService.java
│       ├── repository/MemberRepository.java
│       ├── entity/Member.java
│       └─ dto/MemberDto.java
└─ resources/application.yml
```

text

## Kotlin

```
member-kotlin/
├─ build.gradle.kts
├─ settings.gradle.kts
├─ gradlew / gradlew.bat
├─ gradle/wrapper/
└─ src/main/
    ├─ kotlin/com/example/member/
    │   ├─ MemberApplication.kt
    │   ├─ controller/MemberController.kt
    │   ├─ service/MemberService.kt
    │   ├─ repository/MemberRepository.kt
    │   ├─ entity/Member.kt
    │   └─ dto/MemberDto.kt
    └─ resources/application.yml
```

text

## 3. MariaDB 준비

### DB 생성

```
CREATE DATABASE memberdb CHARACTER SET utf8mb4;
USE memberdb;
```

sql

예제에서는 spring.jpa.hibernate.ddl-auto=update를 사용하여 Entity를 기준으로 테이블을 생성·갱신함. 운영환경에서는 스키마 마이그레이션 도구 사용을 검토하는 것이 좋음.

## 4. Entity

### Java - Member.java

```
@Entity
@Table(name = "members")
@Getter @Setter @NoArgsConstructor
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String email;
    private Integer age;
}
```

java

## Kotlin - Member.kt

```
@Entity
@Table(name = "members")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,

    @Column(nullable = false)
    var name: String = "",

    var email: String = "",
    var age: Int = 0
)
```

kotlin

Entity는 Java/Kotlin 객체와 DB 테이블을 연결함. @Id는 기본키, @GeneratedValue는 키 자동 생성 전략을 의미함.

## 5. DTO

### Java - MemberDto.java

```
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class MemberDto {
    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

java

### Kotlin - MemberDto.kt

```
data class MemberDto(
    val id: Long? = null,
    val name: String,
    val email: String,
    val age: Int
)
```

kotlin

DTO는 API 입출력 데이터를 전달하는 객체임. Entity를 외부 API에 직접 노출하지 않고 필요한 필드만 전달하는 구조를 만들 수 있음.

## 6. Repository

### Java

```
public interface MemberRepository
    extends JpaRepository<Member, Long> {
}
```

java

## Kotlin

```
interface MemberRepository :  
    JpaRepository<Member, Long>
```

kotlin

JpaRepository가 save(), findAll(), findById(), existsById(), deleteById() 등 기본 CRUD를 제공함.

## 7. Service — CRUD 핵심

### Java - MemberService.java

java

```
@Service
@RequiredArgsConstructor
public class MemberService {
    private final MemberRepository repository;

    public MemberDto create(MemberDto dto) {
        Member m = new Member();
        m.setName(dto.getName());
        m.setEmail(dto.getEmail());
        m.setAge(dto.getAge());
        return toDto(repository.save(m));
    }

    public List<MemberDto> findAll() {
        return repository.findAll().stream()
            .map(this::toDto).toList();
    }

    public MemberDto findById(Long id) {
        Member m = repository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("회원이 없습니다."));
        return toDto(m);
    }

    public MemberDto update(Long id, MemberDto dto) {
        Member m = repository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("회원이 없습니다."));
        m.setName(dto.getName());
        m.setEmail(dto.getEmail());
        m.setAge(dto.getAge());
        return toDto(repository.save(m));
    }

    public void delete(Long id) {
        if (!repository.existsById(id))
            throw new RuntimeException("회원이 없습니다.");
        repository.deleteById(id);
    }

    private MemberDto toDto(Member m) {
        return new MemberDto(
            m.getId(), m.getName(), m.getEmail(), m.getAge());
    }
}
```

## Kotlin - MemberService.kt

kotlin

```
@Service
class MemberService(
    private val repository: MemberRepository
) {
    fun create(dto: MemberDto): MemberDto =
        repository.save(
            Member(name=dto.name, email=dto.email, age=dto.age)
        ).toDto()

    fun findAll(): List<MemberDto> =
        repository.findAll().map { it.toDto() }

    fun findById(id: Long): MemberDto =
        repository.findById(id)
            .orElseThrow { RuntimeException("회원이 없습니다.") }
            .toDto()

    fun update(id: Long, dto: MemberDto): MemberDto {
        val m = repository.findById(id)
            .orElseThrow { RuntimeException("회원이 없습니다.") }
        m.name = dto.name
        m.email = dto.email
        m.age = dto.age
        return repository.save(m).toDto()
    }

    fun delete(id: Long) {
        if (!repository.existsById(id))
            throw RuntimeException("회원이 없습니다.")
        repository.deleteById(id)
    }

    private fun Member.toDto() =
        MemberDto(id, name, email, age)
}
```

## 8. REST Controller

---

### Java - MemberController.java

```
@RestControllerjava
@RequestMapping("/api/members")
@RequiredArgsConstructor
public class MemberController {
    private final MemberService service;

    @PostMapping
    public MemberDto create(@RequestBody MemberDto dto) {
        return service.create(dto);
    }

    @GetMapping
    public List<MemberDto> findAll() {
        return service.findAll();
    }

    @GetMapping("/{id}")
    public MemberDto findById(@PathVariable Long id) {
        return service.findById(id);
    }

    @PutMapping("/{id}")
    public MemberDto update(
        @PathVariable Long id,
        @RequestBody MemberDto dto) {
        return service.update(id, dto);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

## Kotlin - MemberController.kt

```
@RestController
@RequestMapping("/api/members")
class MemberController(
    private val service: MemberService
) {
    @PostMapping
    fun create(@RequestBody dto: MemberDto) =
        service.create(dto)

    @GetMapping
    fun findAll() = service.findAll()

    @GetMapping("/{id}")
    fun findById(@PathVariable id: Long) =
        service.findById(id)

    @PutMapping("/{id}")
    fun update(
        @PathVariable id: Long,
        @RequestBody dto: MemberDto
    ) = service.update(id, dto)

    @DeleteMapping("/{id}")
    fun delete(@PathVariable id: Long) =
        service.delete(id)
}
```

kotlin

## 9. application.yml

### MariaDB 연결

```
spring:
  datasource:
    url: jdbc:mariadb://localhost:3306/memberdb
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true

server:
  port: 8080
```

yaml

## 10. 실제 CRUD 테스트

### CREATE

POST http://localhost:8080/api/members

text

```
{
  "name": "홍길동",
  "email": "hong@test.com",
  "age": 35
}
```

### READ

GET http://localhost:8080/api/members

text

GET http://localhost:8080/api/members/1

### UPDATE

PUT http://localhost:8080/api/members/1

text

```
{
  "name": "김길동",
  "email": "kim@test.com",
  "age": 40
}
```

### DELETE

DELETE http://localhost:8080/api/members/1

sql

## 11. Java vs Kotlin 상세 비교

| 항목             | Java                         | Kotlin             |
|----------------|------------------------------|--------------------|
| Entity         | 명시적이며 코드가 다소 김               | 프로퍼티로 간결           |
| DTO            | 클래스/record/Lombok            | data class가 편리     |
| Repository     | 매우 간단                        | 매우 간단              |
| Service        | Stream·getter/setter 등 코드 증가 | 컬렉션 함수와 프로퍼티로 간결   |
| Controller     | 명시적                          | 표현식 함수로 간결         |
| Null 처리        | 주의 필요                        | 언어 차원의 nullable 타입 |
| JPA 생태계        | 매우 성숙                        | Spring 지원 우수       |
| 학습자료           | 매우 많음                        | 많음                 |
| 기존 기업 시스템      | 강점                           | 점차 증가              |
| 신규 Spring 프로젝트 | 매우 좋음                        | 매우 좋음              |
| 코드량            | 상대적으로 많음                     | 상대적으로 적음           |

## 12. 계층별 역할 비교

| 계층            | 역할              | 예제                             |
|---------------|-----------------|--------------------------------|
| Controller    | HTTP 요청/응답      | POST, GET, PUT, DELETE         |
| DTO           | API 데이터 전달      | MemberDto                      |
| Service       | 업무 로직 및 트랜잭션 중심 | create, update 등               |
| Entity        | DB 테이블 매핑       | Member                         |
| Repository    | DB 접근           | JpaRepository                  |
| JPA/Hibernate | 객체와 SQL 연결      | INSERT/SELECT/UPDATE/DELETE 생성 |
| Database      | 데이터 저장          | MariaDB/PostgreSQL/Oracle      |

## 13. 실무형으로 발전시키는 순서

기본 CRUD 완성 → Request DTO / Response DTO 분리 → Bean Validation → 전역 예외처리 → HTTP 상태코드 정리 → 검색 → 페이징/정렬 → 트랜잭션 → 테스트 → Swagger/OpenAPI 순으로 확장하면 이해하기 쉬움.

실무에서는 비밀번호나 운영 DB 접속정보를 소스에 직접 저장하지 않고 환경변수나 비밀번호관리 방식을 사용하는 것이 바람직함.

## 14. 최종 핵심 암기

등록: POST → Controller → Service → DTO→Entity → repository.save() → INSERT

조회: GET → Controller → Service → repository.find...() → SELECT → Entity→DTO

수정: PUT → 기존 Entity 조회 → 값 변경 → save() → UPDATE

삭제:

DELETE → 존재 확인 → deleteById() → DELETE

sql

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER\_IP)·계정(devuser)·비밀번호(CHANGE\_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.